

Jolt Perf Report

Modular stack vs. jolt-core, sha2-chain log_t=16

2026-04-20

Headline

Metric	Core	Modular	Ratio
prove_ms	4,384.2	79,890.6	18.22x
verify_ms	83.8	0.8	0.01x
peak RSS (MB)	1,553.0	5,164.0	3.33x
proof bytes	80,244.0	80,090.0	1.00x

Overall modular is **18.22×** slower than core on **prove**. Both stacks produce valid proofs over the same workload (sha2-chain, -num-iters 16 -log-t 16, $\sim 2^{16} = 65,536$ cycles).

Trace self-time is wall-time $\times$ parallelism; treat as CPU-time breakdown, not wall-time budget.

1 Per-stage CPU breakdown

Spans are bucketed by coarse stage labels. Self_ms is the time spent in the span excluding its instrumented children. The sum across stages $\neq$ wall time: on parallel sections, self_ms accumulates across worker threads.

1.1 Core stack

Stage	self_ms	total_ms	span calls	top span
pcs_misc	5,424 ms	12.4 s	456	multi_pair_g2_setup_parallel
booleanity_sc	2,105 ms	2,114 ms	41	BooleanitySumcheckProver::ingest_challenge
commitment	1,552 ms	7,596 ms	2.7k	DoryCommitmentScheme::compute_tier1_commi...
witness_gen	819 ms	819 ms	1	generate_and_commit_witness_polynomials
other	769 ms	2,450 ms	13.9k	vector_scalar_mul_add_gamma_g1_online
host	432 ms	438 ms	5	Program::build_with_features
read_raf_sc	84.1 ms	156 ms	385	InstructionReadRafSumcheckProver::compute...
poly_ops	81.1 ms	112 ms	5.3k	MultilinearPolynomial::bind_parallel
instruction_ra_sc	47.1 ms	66.3 ms	33	InstructionRaSumcheckProver::compute_message
registers_sc	38.3 ms	51.0 ms	115	RegistersReadWriteCheckingProver::compute...
r1cs_sc	32.6 ms	51.6 ms	155	InstructionInputSumcheckProver::compute_me...
ram_sc	14.5 ms	20.2 ms	126	RamRaVirtualSumcheckProver::compute_message
stage-root	5.2 ms	8,655 ms	10	prove_stage5
preprocess	140 μ s	344 ms	4	BytecodePreprocessing::preprocess
opening_proof	13 μ s	770 ms	2	DoryCommitmentScheme::prove

1.2 Modular stack

Stage	self_ms	total_ms	span calls	top span
other	50.1 s	213.0 s	715	CpuBackend::gruen_segmented_reduce
cpu.reduce	28.8 s	31.9 s	2.4k	reduce_dense
cpu.interpolate	19.2 s	19.2 s	17.4k	interpolate_inplace
pcs.dory	16.7 s	39.7 s	43.3k	multi_pair_g2_setup_parallel
op.commit	4,294 ms	23.7 s	45	Commit
host	829 ms	836 ms	4	Program::build_with_features
infra.module_boundary	250 ms	25.4 s	599	r1cs::Az
op.eq_table	129 ms	15.0 s	150	mb::EqProject
sumcheck.read_raf	98.8 ms	98.8 ms	256	ReadCheckingReduce
poly_ops	74.2 ms	74.2 ms	94	EqPolynomial::evals
op.bind	23.7 ms	19.3 s	495	Bind
op.batch	18.4 ms	18.4 ms	1.4k	BatchRoundBegin
op.materialize	13.5 ms	25.3 s	294	MaterializeRA
op.transcript	9.8 ms	9.8 ms	602	AbsorbRoundPoly
op.reduce	1.6 ms	6,600 ms	556	InstanceReduce
preprocess	455 µs	456 µs	2	BytecodePreprocessing::preprocess
op.segmented_reduce	447 µs	21.0 s	32	InstanceSegmentedReduce
misc.compute	86 µs	86 µs	338	ComputePower

2 Top spans — core

#	span	self_ms	total_ms	count	mean_us
1	multi_pair_g2_setup_parallel	4,691.1	5,522.6	62	89,074.8
2	DoryCommitmentScheme::compute_tier1_commitment_onehot	1,193.8	1,230.5	2.6k	480.7
3	BooleanitySumcheckProver::ingest_challenge	994.7	994.7	20	49,734.7
4	BooleanitySumcheckProver::compute_message	958.7	958.7	20	47,934.3
5	generate_and_commit_witness_polynomials	819.2	819.2	1	819,185.9
6	Program::build_with_features	423.5	423.5	2	211,732.6
7	vector_scalar_mul_add_gamma_g1_online	394.6	394.6	42	9,394.5
8	DoryCommitmentScheme::setup_prover	343.5	343.5	1	343,545.3
9	multi_pair_parallel	332.3	332.3	51	6,516.4
10	msm_i128	168.0	168.0	128	1,312.5
11	BooleanitySumcheckProver::initialize	151.7	160.6	1	160,558.9
12	msm_field_elements	149.7	149.7	65	2,303.7
13	DoryProverState::apply_first_challenge	109.3	109.3	10	10,933.7
14	multi_pair_g1_setup_parallel	81.3	81.3	18	4,514.9
15	DoryVerifierState::process_round	59.2	59.2	10	5,916.9
16	DoryProverState::apply_second_challenge	54.0	54.0	10	5,397.8
17	InstructionRaSumcheckProver::compute_message	42.2	42.2	16	2,634.7
18	InstructionReadRafSumcheckProver::compute_message	37.3	37.3	144	259.2
19	create_evaluation_proof	36.0	705.9	1	705,854.0
20	RegistersReadWriteCheckingProver::compute_message	32.3	32.3	23	1,404.2
21	MultilinearPolynomial::bind_parallel	21.0	29.8	2.1k	13.9
22	BytecodeReadRafSumcheckProver::compute_message	16.6	16.6	30	552.9
23	RaPolynomialRound3::bind	16.4	16.4	42	391.5
24	DoryProverState::compute_first_message	15.6	240.0	10	24,003.7
25	bound_poly_var_top_zero_optimized	15.4	15.4	13.1k	1.2
26	InstructionInputSumcheckProver::compute_message	14.3	14.3	16	893.4
27	InstructionReadRafSumcheckProver::ingest_challenge	13.8	49.9	144	346.8

#	span	self_ms	total_ms	count	mean_us
28	materialized_vector_matrix_product	12.2	12.2	1	12,220.0
29	EqPolynomial::evals_parallel	9.0	9.0	5	1,809.6
30	ReadWriteMatrixCycleMajor::bind	9.0	9.0	32	280.3
31	trace	8.5	9.2	2	4,596.5
32	build_streaming_rlc	8.5	409.6	1	409,617.8
33	CompactPolynomial::bind	8.3	8.3	247	33.7
34	PrefixSuffix::init_Q_raf	8.0	8.0	16	499.0
35	InstructionReadRafProver::init_suffix_polys	7.8	7.8	16	485.1
36	shared_ra_polys::compute_all_G	6.8	6.8	1	6,751.3
37	new_streaming	6.3	6.3	1	6,275.3
38	DoryCommitmentScheme::combine_commitments	6.2	12.4	2	6,196.8
39	DoryProverState::compute_second_message	6.1	244.8	10	24,475.9
40	Update u_evals	5.8	5.8	15	386.3

3 Top spans — modular

#	span	self_ms	total_ms	count	mean_us
1	reduce_dense	28,770.7	31,892.3	2.4k	13,439.6
2	interpolate_inplace	19,204.4	19,204.4	17.4k	1,102.7
3	CpuBackend::gruen_segmented_reduce	18,137.6	18,137.6	16	1,133,600.1
4	multi_pair_g2_setup_parallel	12,443.2	16,542.7	62	266,818.2
5	CpuBackend::eq_project	9,815.8	9,880.5	82	120,494.3
6	derived::ram_val	6,876.6	6,876.6	1	6,876,555.8
7	derived::ram_ra_indicator	4,950.5	4,950.5	2	2,475,245.6
8	G1::msm	3,061.3	3,061.3	43.0k	71.1
9	Commit	2,974.9	2,975.2	3	991,746.1
10	CpuBackend::segmented_reduce	2,878.9	2,878.9	16	179,932.7
11	derived::ram_combined_ra	2,596.8	2,596.8	1	2,596,761.1
12	ReduceOpenings	1,592.0	2,287.5	1	2,287,525.9
13	DoryScheme::commit	1,319.6	20,743.9	42	493,901.4
14	Program::build_with_features	823.9	823.9	2	411,948.0
15	multi_pair_parallel	731.7	731.7	50	14,633.6
16	rlc_combine	695.5	695.5	1	695,522.5
17	DoryProverState::apply_first_challenge	616.5	616.5	10	61,645.1
18	CpuBackend::transpose_from_host	502.9	502.9	40	12,573.6
19	DoryScheme::setup_prover	375.1	375.1	1	375,058.7
20	DoryProverState::apply_second_challenge	302.5	302.5	10	30,246.7
21	create_evaluation_proof	276.9	1,622.1	1	1,622,126.7
22	mb::EqProject	128.4	14,959.5	82	182,433.4
23	reduce_domain	125.9	125.9	2	62,953.0
24	r1cs::Az	120.2	120.2	1	120,194.2
25	SuffixScatter	120.1	120.1	16	7,508.9
26	QBufferScatter	97.8	97.8	16	6,111.8
27	derived::reg_val	97.4	97.4	1	97,428.8
28	ReadCheckingReduce	96.7	96.7	128	755.6
29	EqPolynomial::evals	74.2	74.2	94	789.8
30	G2::msm	69.3	69.3	30	2,310.3
31	r1cs::Bz	68.2	68.2	1	68,229.2
32	multi_pair_g1_setup_parallel	65.0	65.0	18	3,612.9
33	LagrangeProject	58.5	58.5	2	29,251.1

#	span	self_ms	total_ms	count	mean_us
34	RegroupConstraints	57.1	245.6	1	245,591.5
35	r1cs::Variable	54.9	54.9	51	1,075.9
36	CpuBackend::eq_table	49.8	49.8	68	732.1
37	derived::reg_rs2_ra	31.6	31.6	1	31,597.8
38	derived::reg_rs1_ra	27.2	27.2	1	27,242.5
39	derived::reg_rd_wa	26.5	26.5	1	26,515.2
40	WeightedSum	24.4	24.6	5	4,924.1

4 Spans present in both stacks

These spans have the same name in both the core and modular traces — direct apples-to-apples comparison.
Ratio = modular / core self_ms.

span	core self_ms	core count	mod self_ms	mod count	ratio
multi_pair_g2_setup_parallel	4,691.1	62	12,443.2	62	2.65x
Program::build_with_features	423.5	2	823.9	2	1.95x
multi_pair_parallel	332.3	51	731.7	50	2.20x
DoryProverState::apply_first_challenge	109.3	10	616.5	10	5.64x
DoryProverState::apply_second_challenge	54.0	10	302.5	10	5.60x
create_evaluation_proof	36.0	1	276.9	1	7.70x
multi_pair_g1_setup_parallel	81.3	18	65.0	18	0.80x
EqPolynomial::evals	2.7	128	74.2	94	27.10x
DoryProverState::compute_first_message	15.6	10	1.5	10	0.10x
trace	8.5	2	5.2	1	0.61x
DoryProverState::compute_second_message	6.1	10	0.3	10	0.05x

5 Modular-only spans (infrastructure tax)

Spans that appear only in the modular trace and contribute ≥ 50 ms of self-time. These are candidates for structural overhead the modular stack pays that the core stack does not.

#	span	self_ms	total_ms	count	mean_us
1	reduce_dense	28,770.7	31,892.3	2.4k	13,439.6
2	interpolate_inplace	19,204.4	19,204.4	17.4k	1,102.7
3	CpuBackend::gruen_segmented_reduce	18,137.6	18,137.6	16	1,133,600.1
4	CpuBackend::eq_project	9,815.8	9,880.5	82	120,494.3
5	derived::ram_val	6,876.6	6,876.6	1	6,876,555.8
6	derived::ram_ra_indicator	4,950.5	4,950.5	2	2,475,245.6
7	G1::msm	3,061.3	3,061.3	43.0k	71.1
8	Commit	2,974.9	2,975.2	3	991,746.1
9	CpuBackend::segmented_reduce	2,878.9	2,878.9	16	179,932.7
10	derived::ram_combined_ra	2,596.8	2,596.8	1	2,596,761.1
11	ReduceOpenings	1,592.0	2,287.5	1	2,287,525.9
12	DoryScheme::commit	1,319.6	20,743.9	42	493,901.4
13	rlc_combine	695.5	695.5	1	695,522.5
14	CpuBackend::transpose_from_host	502.9	502.9	40	12,573.6
15	DoryScheme::setup_prover	375.1	375.1	1	375,058.7
16	mb::EqProject	128.4	14,959.5	82	182,433.4
17	reduce_domain	125.9	125.9	2	62,953.0
18	r1cs::Az	120.2	120.2	1	120,194.2
19	SuffixScatter	120.1	120.1	16	7,508.9

#	span	self_ms	total_ms	count	mean_us
20	QBufferScatter	97.8	97.8	16	6,111.8
21	derived::reg_val	97.4	97.4	1	97,428.8
22	ReadCheckingReduce	96.7	96.7	128	755.6
23	G2::msm	69.3	69.3	30	2,310.3
24	r1cs::Bz	68.2	68.2	1	68,229.2
25	LagrangeProject	58.5	58.5	2	29,251.1
26	RegroupConstraints	57.1	245.6	1	245,591.5
27	r1cs::Variable	54.9	54.9	51	1,075.9

6 Core-only spans (differences in what modular instruments)

Spans that only appear in the core trace. Either modular does equivalent work but under a different span name (re-buckets into `Op::*`), or the work is simply absent on the modular path.

#	span	self_ms	total_ms	count	mean_us
1	DoryCommitmentScheme::compute_tier1_commitment_on...	1,193.8	1,230.5	2.6k	480.7
2	BooleanitySumcheckProver::ingest_challenge	994.7	994.7	20	49,734.7
3	BooleanitySumcheckProver::compute_message	958.7	958.7	20	47,934.3
4	generate_and_commit_witness_polynomials	819.2	819.2	1	819,185.9
5	vector_scalar_mul_add_gamma_g1_online	394.6	394.6	42	9,394.5
6	DoryCommitmentScheme::setup_prover	343.5	343.5	1	343,545.3
7	msm_i128	168.0	168.0	128	1,312.5
8	BooleanitySumcheckProver::initialize	151.7	160.6	1	160,558.9
9	msm_field_elements	149.7	149.7	65	2,303.7
10	DoryVerifierState::process_round	59.2	59.2	10	5,916.9
11	InstructionRaSumcheckProver::compute_message	42.2	42.2	16	2,634.7
12	InstructionReadRafSumcheckProver::compute_message	37.3	37.3	144	259.2
13	RegistersReadWriteCheckingProver::compute_message	32.3	32.3	23	1,404.2
14	MultilinearPolynomial::bind_parallel	21.0	29.8	2.1k	13.9
15	BytecodeReadRafSumcheckProver::compute_message	16.6	16.6	30	552.9
16	RaPolynomialRound3::bind	16.4	16.4	42	391.5
17	bound_poly_var_top_zero_optimized	15.4	15.4	13.1k	1.2
18	InstructionInputSumcheckProver::compute_message	14.3	14.3	16	893.4
19	InstructionReadRafSumcheckProver::ingest_challenge	13.8	49.9	144	346.8
20	materialized_vector_matrix_product	12.2	12.2	1	12,220.0

7 Hot-path call tree

Top sub-spans under the dominant parents. Columns show the span's own `total_ms` and `self_ms`, and how many times it was called from that parent.

7.1 Core — prove subtree

node	total_ms	self_ms	calls
prove	5,090 ms	134 μ s	2
↪ prove_stage6	2,224 ms	1.3 ms	1
↪ BooleanitySumcheckProver::ingest_challenge	995 ms	995 ms	20
↪ BooleanitySumcheckProver::compute_message	959 ms	959 ms	20
↪ BooleanitySumcheckProver::initialize	161 ms	152 ms	1
↪ EqPolynomial::evals	11.6 ms	2.7 ms	1

node	total_ms	self_ms	calls
↪ InstructionRaSumcheckProver::compute_message	42.2 ms	42.2 ms	16
↪ InstructionRaSumcheckProver::ingest_challenge	19.9 ms	763 µs	16
↪ MultilinearPolynomial::bind_parallel	29.8 ms	21.0 ms	416
↪ RaPolynomialRound3::bind	16.4 ms	16.4 ms	32
↪ RaPolynomialRound2::bind	2.8 ms	2.8 ms	32
↪ RaPolynomialRound1::bind	324 µs	324 µs	32
↪ GruenSplitEqPolynomial::bind	187 µs	187 µs	16
↪ prove_stage8	1,116 ms	45 µs	1
↪ DoryCommitmentScheme::prove	706 ms	7 µs	1
↪ prove	5,090 ms	134 µs	1
↪ build_streaming_rlc	410 ms	8.5 ms	1
↪ DoryCommitmentScheme::combine_hints	395 ms	300 µs	1
↪ new_streaming	6.3 ms	6.3 ms	1
↪ generate_and_commit_witness_polynomials	819 ms	819 ms	1
↪ create_evaluation_proof	706 ms	36.0 ms	1
↪ DoryProverState::compute_second_message	245 ms	6.1 ms	10
↪ BN254::multi_pair	333 ms	245 µs	20
↪ msm_field_elements	150 ms	150 ms	40
↪ DoryProverState::compute_first_message	240 ms	15.6 ms	10
↪ BN254::multi_pair_g2_setup	5,523 ms	418 µs	20
↪ msm_field_elements	150 ms	150 ms	22
↪ BN254::multi_pair_g1_setup	81.3 ms	78 µs	18
↪ msm_field_elements	150 ms	150 ms	3
↪ DoryProverState::apply_first_challenge	109 ms	109 ms	10
↪ DoryProverState::apply_second_challenge	54.0 ms	54.0 ms	10
↪ prove_stage5	104 ms	1.7 ms	1
↪ InstructionReadRafSumcheckProver::ingest_challenge	49.9 ms	13.8 ms	144
↪ MultilinearPolynomial::bind_parallel	29.8 ms	21.0 ms	144
↪ InstructionReadRafProver::init_phase	21.9 ms	234 µs	15
↪ InstructionReadRafProver::init_log_t_rounds	3.5 ms	342 µs	1
↪ update_checkpoints	2.5 ms	2.5 ms	64
↪ GruenSplitEqPolynomial::bind	187 µs	187 µs	16
↪ InstructionReadRafSumcheckProver::compute_message	37.3 ms	37.3 ms	144
↪ InstructionReadRafSumcheckProver::initialize	4.7 ms	175 µs	1
↪ InstructionReadRafProver::init_phase	21.9 ms	234 µs	1
↪ Compute u_evals	1.7 ms	15 µs	1
↪ Extract vectors	1.1 ms	788 µs	1
↪ ExpandingTable::new	571 µs	571 µs	16
↪ Build cycle_data	266 µs	266 µs	1
↪ RegistersValEvaluationSumcheckProver::compute_message	4.1 ms	4.1 ms	16
↪ RamRaClaimReductionSumcheckProver::compute_message	2.3 ms	2.3 ms	16

7.2 Modular — modular_prove subtree

node	total_ms	self_ms	calls
modular_prove	79.9 s	1.4 ms	1
↪ stage	76.9 s	9.1 ms	8
↪ InstanceSegmentedReduce	21.0 s	447 µs	32
↪ CpuBackend::gruen_segmented_reduce	18.1 s	18.1 s	16

node	total_ms	self_ms	calls
↳ CpuBackend::segmented_reduce	2,879 ms	2,879 ms	16
↳ InstanceBind	19.1 s	7.0 ms	319
↳ interpolate_inplace	19.2 s	19.2 s	2978
↳ Materialize	15.5 s	592 µs	197
↳ mb::EqProject	15.0 s	128 ms	78
↳ pm::Derived	14.6 s	364 µs	2
↳ CpuBackend::eq_project	9,881 ms	9,816 ms	82
↳ pm::Witness	443 µs	443 µs	80
↳ mb::Provided	9,690 ms	119 µs	2
↳ pm::Derived	14.6 s	364 µs	15
↳ pm::R1cs	244 ms	262 µs	13
↳ mb::upload	4.7 ms	4.7 ms	38
↳ pm::Witness	443 µs	443 µs	6
↳ pm::Preprocessed	195 µs	195 µs	4
↳ mb::Transpose	503 ms	313 µs	40
↳ CpuBackend::transpose_from_host	503 ms	503 ms	40
↳ pm::Witness	443 µs	443 µs	40
↳ mb::EqTable	50.0 ms	231 µs	61
↳ CpuBackend::eq_table	49.8 ms	49.8 ms	68
↳ mb::EqGather	23.1 ms	23 µs	6
↳ pm::Derived	14.6 s	364 µs	2
↳ CpuBackend::eq_gather	3.3 ms	3.1 ms	6
↳ pm::Preprocessed	195 µs	195 µs	4
↳ mb::EqPushforward	13.8 ms	34 µs	5
↳ CpuBackend::eq_pushforward	13.7 ms	6.2 ms	5
↳ pm::Preprocessed	195 µs	195 µs	5
↳ MaterializeUnlessFresh	9,687 ms	189 µs	46
↳ mb::Provided	9,690 ms	119 µs	36
↳ pm::Derived	14.6 s	364 µs	15
↳ pm::R1cs	244 ms	262 µs	13
↳ mb::upload	4.7 ms	4.7 ms	38
↳ pm::Witness	443 µs	443 µs	6
↳ pm::Preprocessed	195 µs	195 µs	4
↳ InstanceReduce	6,600 ms	1.3 ms	308
↳ reduce_dense	31.9 s	28.8 s	308
↳ reduce_dense	31.9 s	28.8 s	72
↳ ReduceOpenings	2,288 ms	1,592 ms	1
↳ rlc_combine	696 ms	696 ms	1
↳ pm::Witness	443 µs	443 µs	40
↳ Commit	2,975 ms	2,975 ms	3
↳ pm::Witness	443 µs	443 µs	44
↳ Preamble	149 µs	149 µs	1

8 Modular prove — time by top-level op

Children of the single modular_prove root span, ordered by total_ms. This is the most trustworthy wall-time rollup on the modular side because it's explicitly scoped.

op	total_ms	% of prove	calls
stage	76.9 s	96.3%	8
Commit	2,975 ms	3.7%	3
Preamble	149 μ s	0.0%	1

8.1 Modular — stage subtree (8 stage invocations)

node	total_ms	self_ms	calls
stage	76.9 s	9.1 ms	8
$\hookrightarrow$ InstanceSegmentedReduce	21.0 s	447 μ s	32
$\hookrightarrow$ CpuBackend::gruen_segmented_reduce	18.1 s	18.1 s	16
$\hookrightarrow$ CpuBackend::segmented_reduce	2,879 ms	2,879 ms	16
$\hookrightarrow$ InstanceBind	19.1 s	7.0 ms	319
$\hookrightarrow$ interpolate_inplace	19.2 s	19.2 s	2978
$\hookrightarrow$ Materialize	15.5 s	592 μ s	197
$\hookrightarrow$ mb::EqProject	15.0 s	128 ms	78
$\hookrightarrow$ pm::Derived	14.6 s	364 μ s	2
$\hookrightarrow$ derived::ram_val	6,877 ms	6,877 ms	1
$\hookrightarrow$ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
$\hookrightarrow$ derived::ram_combined_ra	2,597 ms	2,597 ms	1
$\hookrightarrow$ derived::reg_val	97.4 ms	97.4 ms	1
$\hookrightarrow$ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
$\hookrightarrow$ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
$\hookrightarrow$ CpuBackend::eq_project	9,881 ms	9,816 ms	82
$\hookrightarrow$ EqPolynomial::evals	74.2 ms	74.2 ms	82
$\hookrightarrow$ pm::Witness	443 μ s	443 μ s	80
$\hookrightarrow$ mb::Provided	9,690 ms	119 μ s	2
$\hookrightarrow$ pm::Derived	14.6 s	364 μ s	15
$\hookrightarrow$ derived::ram_val	6,877 ms	6,877 ms	1
$\hookrightarrow$ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
$\hookrightarrow$ derived::ram_combined_ra	2,597 ms	2,597 ms	1
$\hookrightarrow$ derived::reg_val	97.4 ms	97.4 ms	1
$\hookrightarrow$ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
$\hookrightarrow$ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
$\hookrightarrow$ pm::R1cs	244 ms	262 μ s	13
$\hookrightarrow$ r1cs::Az	120 ms	120 ms	1
$\hookrightarrow$ r1cs::Bz	68.2 ms	68.2 ms	1
$\hookrightarrow$ r1cs::Variable	54.9 ms	54.9 ms	51
$\hookrightarrow$ mb::upload	4.7 ms	4.7 ms	38
$\hookrightarrow$ pm::Witness	443 μ s	443 μ s	6
$\hookrightarrow$ pm::Preprocessed	195 μ s	195 μ s	4
$\hookrightarrow$ mb::Transpose	503 ms	313 μ s	40
$\hookrightarrow$ CpuBackend::transpose_from_host	503 ms	503 ms	40
$\hookrightarrow$ pm::Witness	443 μ s	443 μ s	40
$\hookrightarrow$ mb::EqTable	50.0 ms	231 μ s	61
$\hookrightarrow$ CpuBackend::eq_table	49.8 ms	49.8 ms	68
$\hookrightarrow$ mb::EqGather	23.1 ms	23 μ s	6
$\hookrightarrow$ pm::Derived	14.6 s	364 μ s	2
$\hookrightarrow$ derived::ram_val	6,877 ms	6,877 ms	1
$\hookrightarrow$ derived::ram_ra_indicator	4,950 ms	4,950 ms	2

node	total_ms	self_ms	calls
↳ derived::ram_combined_ra	2,597 ms	2,597 ms	1
↳ derived::reg_val	97.4 ms	97.4 ms	1
↳ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
↳ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
↳ CpuBackend::eq_gather	3.3 ms	3.1 ms	6
↳ EqPolynomial::evals	74.2 ms	74.2 ms	6
↳ pm::Preprocessed	195 µs	195 µs	4
↳ mb::EqPushforward	13.8 ms	34 µs	5
↳ CpuBackend::eq_pushforward	13.7 ms	6.2 ms	5
↳ EqPolynomial::evals	74.2 ms	74.2 ms	5
↳ pm::Preprocessed	195 µs	195 µs	5
↳ MaterializeUnlessFresh	9,687 ms	189 µs	46
↳ mb::Provided	9,690 ms	119 µs	36
↳ pm::Derived	14.6 s	364 µs	15
↳ derived::ram_val	6,877 ms	6,877 ms	1
↳ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
↳ derived::ram_combined_ra	2,597 ms	2,597 ms	1
↳ derived::reg_val	97.4 ms	97.4 ms	1
↳ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
↳ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
↳ pm::R1cs	244 ms	262 µs	13
↳ r1cs::Az	120 ms	120 ms	1
↳ r1cs::Bz	68.2 ms	68.2 ms	1
↳ r1cs::Variable	54.9 ms	54.9 ms	51
↳ mb::upload	4.7 ms	4.7 ms	38
↳ pm::Witness	443 µs	443 µs	6
↳ pm::Preprocessed	195 µs	195 µs	4
↳ InstanceReduce	6,600 ms	1.3 ms	308
↳ reduce_dense	31.9 s	28.8 s	308
↳ reduce_dense	31.9 s	28.8 s	72
↳ ReduceOpenings	2,288 ms	1,592 ms	1
↳ rlc_combine	696 ms	696 ms	1
↳ pm::Witness	443 µs	443 µs	40

8.2 Modular — Materialize / mb::* family

The materialize-on-demand path for derived polynomials. `mb::EqProject` dominates because it runs every sumcheck round.

node	total_ms	self_ms	calls
Materialize	15.5 s	592 µs	197
↳ mb::EqProject	15.0 s	128 ms	78
↳ pm::Derived	14.6 s	364 µs	2
↳ derived::ram_val	6,877 ms	6,877 ms	1
↳ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
↳ derived::ram_combined_ra	2,597 ms	2,597 ms	1
↳ derived::reg_val	97.4 ms	97.4 ms	1
↳ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
↳ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
↳ derived::reg_rd_wa	26.5 ms	26.5 ms	1
↳ derived::ram_gather_index	19.2 ms	19.2 ms	1

node	total_ms	self_ms	calls
↪ derived::lookup_table_flag	7.3 ms	7.3 ms	41
↪ derived::product_left	1.4 ms	1.4 ms	1
↪ CpuBackend::eq_project	9,881 ms	9,816 ms	82
↪ EqPolynomial::evals	74.2 ms	74.2 ms	82
↪ pm::Witness	443 µs	443 µs	80
↪ mb::Provided	9,690 ms	119 µs	2
↪ pm::Derived	14.6 s	364 µs	15
↪ derived::ram_val	6,877 ms	6,877 ms	1
↪ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
↪ derived::ram_combined_ra	2,597 ms	2,597 ms	1
↪ derived::reg_val	97.4 ms	97.4 ms	1
↪ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
↪ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
↪ derived::reg_rd_wa	26.5 ms	26.5 ms	1
↪ derived::ram_gather_index	19.2 ms	19.2 ms	1
↪ derived::lookup_table_flag	7.3 ms	7.3 ms	41
↪ derived::product_left	1.4 ms	1.4 ms	1
↪ pm::R1cs	244 ms	262 µs	13
↪ r1cs::Az	120 ms	120 ms	1
↪ r1cs::Bz	68.2 ms	68.2 ms	1
↪ r1cs::Variable	54.9 ms	54.9 ms	51
↪ mb::upload	4.7 ms	4.7 ms	38
↪ pm::Witness	443 µs	443 µs	6
↪ pm::Preprocessed	195 µs	195 µs	4
↪ mb::Transpose	503 ms	313 µs	40
↪ CpuBackend::transpose_from_host	503 ms	503 ms	40
↪ pm::Witness	443 µs	443 µs	40
↪ mb::EqTable	50.0 ms	231 µs	61
↪ CpuBackend::eq_table	49.8 ms	49.8 ms	68
↪ mb::EqGather	23.1 ms	23 µs	6
↪ pm::Derived	14.6 s	364 µs	2
↪ derived::ram_val	6,877 ms	6,877 ms	1
↪ derived::ram_ra_indicator	4,950 ms	4,950 ms	2
↪ derived::ram_combined_ra	2,597 ms	2,597 ms	1
↪ derived::reg_val	97.4 ms	97.4 ms	1
↪ derived::reg_rs2_ra	31.6 ms	31.6 ms	1
↪ derived::reg_rs1_ra	27.2 ms	27.2 ms	1
↪ derived::reg_rd_wa	26.5 ms	26.5 ms	1
↪ derived::ram_gather_index	19.2 ms	19.2 ms	1
↪ derived::lookup_table_flag	7.3 ms	7.3 ms	41
↪ derived::product_left	1.4 ms	1.4 ms	1
↪ CpuBackend::eq_gather	3.3 ms	3.1 ms	6
↪ EqPolynomial::evals	74.2 ms	74.2 ms	6
↪ pm::Preprocessed	195 µs	195 µs	4
↪ mb::EqPushforward	13.8 ms	34 µs	5
↪ CpuBackend::eq_pushforward	13.7 ms	6.2 ms	5
↪ EqPolynomial::evals	74.2 ms	74.2 ms	5
↪ pm::Preprocessed	195 µs	195 µs	5
↪ mb::EqPlusOneTable	5.4 ms	8 µs	2
↪ CpuBackend::eq_plus_one_table	5.4 ms	5.4 ms	2
↪ mb::LtTable	3.4 ms	14 µs	2
↪ CpuBackend::lt_table	3.4 ms	3.4 ms	2

node	total_ms	self_ms	calls
↪ mb::ScaleByChallenge	291 μ s	2 μ s	1
↪ CpuBackend::scale_from_host	289 μ s	289 μ s	1
↪ pm::Preprocessed	195 μ s	195 μ s	1

8.3 Modular — pm::Derived (lazy witness poly materialization)

Every row is a polynomial that the modular stack derives on-demand during prove. Core materializes these once in generate_and_commit_witness_polynomials (~819 ms total).

derived poly	total_ms	calls
derived::ram_val	6,877 ms	1
derived::ram_ra_indicator	4,950 ms	2
derived::ram_combined_ra	2,597 ms	1
derived::reg_val	97.4 ms	1
derived::reg_rs2_ra	31.6 ms	1
derived::reg_rs1_ra	27.2 ms	1
derived::reg_rd_wa	26.5 ms	1
derived::ram_gather_index	19.2 ms	1
derived::lookup_table_flag	7.3 ms	41
derived::product_left	1.4 ms	1
derived::product_right	1.2 ms	1
derived::extract_column	591 μ s	2
derived::rd_gather_index	454 μ s	1
derived::hamming_weight	410 μ s	1
derived::instruction_raf_flag	168 μ s	1
derived::ram_val_final	63 μ s	1
derived::iflag_right_is_imm	0 μ s	1
derived::iflag_left_is_rs1	0 μ s	1
derived::iflag_right_is_rs2	0 μ s	1
derived::iflag_noop	0 μ s	1
derived::iflag_left_is_pc	0 μ s	1

9 The three heaviest modular-only kernels

These three spans together account for the majority of the modular overhead. Each is part of the CPU backend's evaluation path for sumcheck kernels.

kernel	self_ms	count	mean_us	parent
reduce_dense	28,770.7	2.4k	13,439.6	InstanceReduce
interpolate_inplace	19,204.4	17.4k	1,102.7	Bind
CpuBackend::gruen_segmented_reduce	18,137.6	16	1,133,600.1	InstanceSegmentedReduce
CpuBackend::segmented_reduce	2,878.9	16	179,932.7	InstanceSegmentedReduce
CpuBackend::eq_project	9,815.8	82	120,494.3	mb::EqProject
CpuBackend::transpose_from_host	502.9	40	12,573.6	mb::Transpose
CpuBackend::eq_table	49.8	68	732.1	mb::EqTable

10 Structural observations

1. **Sumcheck hot path: 46,718 ms (~58% of modular prove).** Modular's InstanceSegmentedReduce + InstanceBind + InstanceReduce trio replaces what core does inside compiled `prove_stageN` closures. The combined overhead vs. core's same-shape sumchecks is $\sim 20\times$. Every round, modular dispatches through CpuBackend kernels that core inlines directly. See the call-count explosion table.
2. **Derived-polynomial materialization: `pm::Derived` = 14,638 ms.** RAM witness polys alone (`ram_val + ram_ra_indicator + ram_combined_ra`) cost 14,424 ms. Core materializes equivalent polynomials once up-front in `generate_and_commit_witness_polynomials` (~ 819 ms). Modular pays this per-use, not per-poly. If these polys are re-derived on each Bind call, caching or bulk-derivation is a big target.
3. **Materialize/EqProject subsystem: 25,219 ms.** The `mb::EqProject` + `CpuBackend::eq_project` path alone is 9,816 ms self. Core's equivalent `EqPolynomial::evals_parallel` is 9.0 ms — a $\sim 1000\times$ gap. Likely re-computing eq tables every round rather than binding incrementally.
4. **PCS commit+open is roughly parity.** Modular Commit = 2,975 ms vs. core's committed polynomial generation = 819 ms. Modular Open (1,623 ms) is already comparable to core `create_evaluation_proof` (706 ms). Dory is not the modular bottleneck; sumcheck and witness-derivation are.
5. **Verify delta is suspicious: core 83.8 ms vs modular 0.8 ms.** Modular's `jolt_verifier::verify` is two orders of magnitude faster, suggesting the Dory PCS opening is *not* being verified on the modular path. Confirm whether the native modular verify is pipelined against the Dory opening proof before declaring modular verify a win.
6. **Memory: 3.3 $\times$.** Modular peak RSS 5.16 GB vs core 1.55 GB. Likely sources: lazily-derived polys held in CpuBackend buffers + per-op output buffers not recycled + multiple copies of eq tables.
7. **Per-round dispatch: `interpolate_inplace` runs 17,416 times in modular vs. not at all as a distinct span in core** (core inlines the interpolation inside its sumcheck `compute_message`). Batching `interpolate_inplace` across kernels in the same round is the prototypical amortization target.

11 Reproduction

```
cargo run --release -p jolt-bench -- \
  --program sha2-chain --num-iters 16 --log-t 16 \
  --iters 1 --warmup 0 --stack core \
  --trace-chrome report_core --json perf/report-core-fresh.json
```

```
cargo run --release -p jolt-bench -- \
  --program sha2-chain --num-iters 16 --log-t 16 \
  --iters 1 --warmup 0 --stack modular \
  --trace-chrome report_modular --json perf/report-modular-fresh.json
```

```
python3 perf/report_tools/analyze_trace.py \
  benchmark-runs/perfetto_traces/report_core.json \
  perf/report_tools/core_spans.json
python3 perf/report_tools/analyze_trace.py \
  benchmark-runs/perfetto_traces/report_modular.json \
  perf/report_tools/modular_spans.json
```

```
python3 perf/report_tools/build_report.py
(cd perf/report_tools && pdflatex -interaction=nonstopmode perf_report.tex)
```